

Visual Labeling Bridge — Installation Guide

A small shim that makes Amplitude's Visual Labeling work when the Browser SDK runs inside nested `iframe` / `frame` elements instead of the top-level page.

If the SDK runs on your top-level page, you do not need this script.

How it works

Visual Labeling relies on `postMessage` traffic between the Amplitude tab and the SDK, plus a working `window.opener` reference on the SDK side. Neither works out of the box when the SDK lives inside a nested frame.

The bridge runs on the top page and does two things:

1. **Inbound:** Listens for `message` events from the Amplitude tab and re-dispatches them into every same-origin nested frame, preserving the original `event.origin` so the SDK's origin check still passes.
2. **Outbound:** Sets `frame.contentWindow.opener` on each same-origin frame to the Amplitude tab, so the SDK's replies (`window.opener.postMessage(...)`) reach Amplitude directly.

A `MutationObserver` rescans the frame tree as frames are added or navigated, so dynamically injected frames are picked up automatically.

Only Amplitude messages are forwarded. The inbound listener checks both `event.source === window.opener` and that `event.origin`'s hostname ends in `.amplitude.com` before relaying.

Messages from any other window or origin are ignored and never reach your frames — the bridge does not act as a generic cross-frame `postMessage` relay.

Requirements

1. **Same-origin frames.** Every frame between the top page and the SDK frame must share the top page's origin (protocol + host + port). Cross-origin frames break the bridge.
2. **Amplitude Browser SDK with `autocapture`** loaded inside the inner frame.
3. **CSP** must allow the script to load and execute on the top page.

Installation

The script must be loaded on the **top-level window only** — the page Amplitude opens when launching Visual Labeling. Do not add it to nested frames. The script will support any nested frame or iframes.

1. Host `vl-bridge.js` on the same origin as your top page (or any host your CSP allows).
2. Add one tag to the top page, near the end of `<body>`:

```
<script src="/path/to/vl-bridge.js"></script>
```

No configuration, globals, or initialization. Load as a classic script — do not wrap it in a module or bundle.

Verification

1. Open the top page directly. Console should be clean; the script no-ops because `window.opener` is null.
2. Launch Visual Labeling from Amplitude targeting your page.
3. The Visual Tagging Selector overlay should appear inside the SDK frame within a few seconds.
4. Select and save an element; confirm it appears in Amplitude.

Diagnostic warnings are prefixed with `[VL bridge]` in the DevTools console of the tab Amplitude opens.

Common issues

Symptom	Likely cause
Overlay never appears.	A frame between the top page and the SDK is cross-origin.
<code>[VL bridge] could not set frame.contentWindow.opener</code>	Cross-origin frame, or a CSP / <code>sandbox</code> attribute blocking access.
Script silent, no warnings.	Loaded inside a frame instead of the top page, or page wasn't opened from Amplitude.
Same-origin but still broken.	Confirm the SDK with autocapture is actually loaded in the inner frame.

Notes

- No network requests, no config, no API keys. Works against US, EU, and staging Amplitude unchanged.
- Does not alter Amplitude tracking outside the Visual Labeling handshake.

Support

Contact your Amplitude rep with: a screenshot/recording, any `[VL bridge]` console warnings, and your frame structure (depth + any cross-origin frames).